

Daftar Isi

1. Ringkasan Eksekutif	3
1.1 Temuan Paling Kritis	3
2. Audit Firmware ESP32 WROOM	4
2.1 Gambaran Umum Arsitektur Firmware	4
2.2 Temuan Keamanan Firmware	4
2.3 Temuan Keselamatan Perangkat Keras	5
2.4 Temuan Kualitas Kode dan Arsitektur	7
3. Audit Backend Google Apps Script	8
3.1 Gambaran Umum Backend	8
3.2 Temuan Keamanan Backend	8
3.3 Temuan Performa dan Arsitektur	10
4. Audit Frontend Next.js / Vercel	11
4.1 Gambaran Umum Frontend	11
4.2 Temuan Keamanan Frontend	11
4.3 Temuan Kualitas Kode dan Performa	13
5. Analisis Integrasi Antar Komponen	14

5.1 Alur Data End-to-End	14
5.2 Temuan Integrasi	14
6. Rekomendasi Perbaikan	15
6.1 Aksi Segera (0-7 Hari)	15
6.2 Prioritas Tinggi (1-4 Minggu)	16
6.3 Prioritas Menengah (1-3 Bulan)	17
7. Kesimpulan	18

1. Ringkasan Eksekutif

Laporan ini merupakan hasil audit komprehensif terhadap Sistem Manajemen Energi Cerdas (SEMS - Smart Energy Management System) yang dikembangkan oleh PT. Jaya Mandiri Smart Energy. Sistem ini terdiri dari empat komponen utama: firmware ESP32 WROOM sebagai perangkat penginderaan dan kontrol perangkat keras, backend Google Apps Script sebagai middleware API, Google Spreadsheet sebagai penyimpanan data dan kredensial, serta frontend Next.js yang di-hosting di Vercel sebagai antarmuka pengguna.

Audit ini mencakup aspek keamanan siber, kualitas kode, arsitektur perangkat lunak, keselamatan perangkat keras (hardware safety), serta integrasi antar komponen. Secara keseluruhan, tim auditor menemukan total **16 temuan KRITIS**, **22 temuan TINGGI**, **26 temuan SEDANG**, dan **20 temuan RENDAH** yang tersebar di seluruh komponen sistem. Temuan-temuan KRITIS meliputi kerentanan keamanan yang dapat dieksploitasi untuk mengakses data sensitif, mengambil alih perangkat, dan berpotensi menyebabkan kerusakan fisik pada perangkat keras baterai.

Komponen	KRITIS	TINGGI	SEDANG	RENDAH	INFO
Firmware ESP32	7	8	10	4	0
Backend GAS	5	5	7	5	0
Frontend Vercel	4	9	18	12	9
Total	16	22	35	21	9

Tabel 1: Ringkasan Temuan Audit per Komponen

1.1 Temuan Paling Kritis

Temuan paling berbahaya yang ditemukan dalam audit ini adalah sebagai berikut. Pertama, **kredensial hardcoded** yang mencakup WiFi password, API token, dan OTA password yang tersimpan dalam plain text di source code firmware. Kedua, **Safety Engine tidak mengambil tindakan protektif** - kondisi overvoltage dan undervoltage hanya mencatat log tanpa memutus relay atau menghentikan charging, yang berpotensi menyebabkan kerusakan baterai LiFePO4 hingga terjadinya kebakaran. Ketiga, **seluruh endpoint GET backend tidak memiliki autentikasi**, sehingga siapa pun yang mengetahui URL Google Apps Script dapat mengakses semua data sistem termasuk token perangkat dan informasi pengguna.

Keempat, **middleware frontend menggunakan cookie statik "sems-auth=1"** yang dapat dengan mudah dipalsukan oleh pengguna melalui browser DevTools, menjadikan seluruh proteksi rute dashboard tidak efektif. Kelima, terdapat **potensi injeksi formula Google Sheets** karena string yang dikontrol pengguna ditulis langsung ke sel spreadsheet tanpa sanitasi, yang dapat menyebabkan eksekusi kode arbitrer dan eksfiltrasi data.

2. Audit Firmware ESP32 WROOM

2.1 Gambaran Umum Arsitektur Firmware

Firmware SEMS dibangun menggunakan framework Arduino dengan PlatformIO sebagai build system. Target hardware adalah ESP32 WROOM DEVKIT yang bertanggung jawab untuk pembacaan sensor (voltage baterai, arus ACS712, suhu/kelembaban SHT31, deteksi gerak PIR, monitoring RTC), kontrol relay untuk pengelolaan beban, pengiriman telemetry ke backend, serta sinkronisasi konfigurasi dari server. Firmware juga mengimplementasikan SOC (State of Charge) calculator, safety engine, automation engine, load shedding engine, dan schedule engine.

Auditor menemukan bahwa proyek ini mengandung **dua basis kode paralel yang tidak sinkron**. File SEMS_Firmware.ino (~1600 baris, monolitik) merupakan kode produksi yang berjalan, sementara direktori sensors/, engines/, dan utils/ berisi versi refactoring berbasis kelas OOP yang **tidak dapat dikompilasi** karena mereferensikan ~40 konstanta yang tidak terdefinisi dan menggunakan EEPROM API yang sudah dihapus di ESP32 Core 3.x. Keberadaan dua versi kode yang tidak kompatibel ini merupakan risiko pemeliharaan yang serius.

2.2 Temuan Keamanan Firmware

ID	Severity	Temuan	File	Dampak
FW-S01	KRITIS	WiFi SSID dan Password Hardcoded	config.h:25-26	Siapa pun dengan akses kode sumber mengetahui kredensial WiFi. Password "1234567890" sangat mudah ditebak.

ID	Severity	Temuan	File	Dampak
FW-S 02	KRITIS	API Token dan Device ID Hardcoded	config.h:36-37	Token "sems_device_token_change_me_2024" adalah placeholder yang tidak pernah diubah. Token ini digunakan untuk autentikasi semua upload telemetry.
FW-S 03	KRITIS	OTA Password Hardcoded	config.h:173	Password OTA terekspos. Versi platformio.ini menggunakan password berbeda ("sems_ota_2024" vs "sems_ota_2026").
FW-S 04	KRITIS	URL Google Apps Script Terekspos	config.h:35	URL backend GAS terekspos dalam kode sumber, memungkinkan penyerang mem-probe endpoint.
FW-S 05	TINGGI	Autentikasi Token Statik	SEMS_Firmware.ino:1249	Single static token tanpa rotasi, tanpa identitas per-perangkat, tanpa mutual TLS.
FW-S 06	TINGGI	Tidak Ada Validasi Input pada Konfigurasi Remote	.ino:1304-1348	Safety thresholds (overvoltage, overcurrent, temperature) dapat diatur ke nilai berbahaya oleh backend yang compromised.
FW-S 07	SEDANG	Tidak Ada Certificate Pinning HTTPS	SEMS_Firmware.ino:1244	Menggunakan WiFiClient default tanpa certificate pinning. MitM dengan CA cert valid dapat menyadap traffic.

Tabel 2: Temuan Keamanan Firmware

2.3 Temuan Keselamatan Perangkat Keras

Temuan paling kritis dari aspek keselamatan perangkat keras adalah bahwa Safety Engine **tidak mengambil tindakan protektif apapun**. Pada kondisi overvoltage (>29.6V pada baterai LiFePO4 8S), firmware hanya mencatat log ke Serial Monitor tanpa memutus relay charging atau melakukan emergency shutdown. Demikian

pula pada kondisi undervoltage (<19.0V), tidak ada aksi emergency load shedding yang diambil. Hal ini berpotensi menyebabkan kerusakan permanen pada sel baterai atau bahkan terjadinya thermal runaway dan kebakaran.

Selain itu, engine load shedding hanya mengimplementasikan pemb shedding biner (matikan semua atau tidak ada), meskipun terdapat threshold bertingkat (TIER1/TIER2/TIER3). Kode mengakui ketidaklengkapan ini dengan komentar "TODO: Implement priority-based load shedding". Pada skenario nyata brownout, beban non-esensial dan beban kenyamanan dimatikan secara bersamaan tanpa prioritas. Monitoring overcurrent menggunakan ACS712 juga tidak diimplementasikan di firmware produksi (.ino), sementara threshold SAFETY_OVERCURRENT_A (31A) didefinisikan tetapi tidak pernah dicek.

ID	Severity	Temuan	File	Dampak
FW-H01	KRITIS	Safety Engine Tidak Mengambil Tindakan Protektif	.ino:825-835	Overvoltage/undervoltage hanya log, tanpa relay cutoff atau emergency shutdown. Risiko thermal runaway.
FW-H02	KRITIS	Load Shedding Biner - Semua Tier Identik	.ino:897-904	Meskipun ada threshold bertingkat, aksi shedding hanya matikan semua relay non-kritis.
FW-H03	TINGGI	Cell Imbalance Hanya Warning Tanpa Aksi	.ino:838-850	Imbalance >300mV pada LiFePO4 mengindikasikan sel rusak, tapi tidak ada pengurangan arus charge/discharge.
FW-H04	TINGGI	Overcurrent Tidak Dicek di Firmware Aktif	.ino safetyEngineLoop()	Threshold SAFETY_OVERCURRENT_A (31A) didefinisikan tapi tidak dicek di .ino yang berjalan.
FW-H05	SEDANG	Suhu Tidak Dimonitor di Firmware Aktif	.ino safetyEngineLoop()	SAFETY_MAX_TEMP_C (50 derajat) didefinisikan tapi tidak dicek. Risiko kerusakan sel dan kebakaran.
FW-H06	SEDANG	Backup Voltage ADC2 Selalu Unavailable	.ino:679-691	ESP32 ADC2 tidak dapat digunakan bersamaan dengan WiFi. Pembacaan voltage backup selalu gagal.

Tabel 3: Temuan Keselamatan Perangkat Keras

2.4 Temuan Kualitas Kode dan Arsitektur

Dari sisi arsitektur, firmware mengalami beberapa masalah struktural yang signifikan. Dual codebase (monolitik .ino vs modular classes) menjamin divergensi dan bug. Struct SensorData yang berukuran ~115 byte merupakan god object yang memegang semua state sistem tanpa enkapsulasi atau access control. Tidak ada mutex atau semaphore untuk shared state, yang berpotensi menyebabkan torn reads jika kode berpindah ke arsitektur multi-task FreeRTOS. Dari sisi kualitas kode, komentar campuran Indonesia dan Inggris mengurangi maintainability, dan tidak ada framework pengujian unit untuk logika safety-critical.

ID	Severity	Temuan	File	Dampak
FW-A01	TINGGI	Dual Codebase - .ino dan Modular Tidak Sinkron	Semua file	Kode modular tidak bisa kompilasi (~40 konstanta hilang, EEPROM API salah). Dua versi menjamin divergensi.
FW-A02	TINGGI	Tidak Ada Mutex untuk Shared State	.ino:70-117	SensorData global ditulis/dibaca oleh banyak engine tanpa proteksi concurrent access.
FW-A03	SEDANG	God Object - SensorData 115 Byte	.ino:70-115	Single struct memegang semua state tanpa enkapsulasi atau change notification.
FW-A04	SEDANG	String Queue Heap Fragmentation	.ino:126-133	50 entry String queue dapat menghabiskan 25KB+ heap. Fragmentasi dapat menyebabkan crash.
FW-A05	SEDANG	EEPROM Wear - SOC Disimpan Setiap 60 Detik	SOC_Calculator.cpp	EEPROM emulation berbasis flash (100K cycles). Write setiap 60 detik = habis dalam ~70 hari.
FW-A06	TINGGI	OTA Password Mismatch config.h vs platformio.ini	config.h:173 / platformio.ini:37	config.h: "sems_ota_2026" vs platformio.ini: "sems_ota_2024". OTA upload akan gagal.

ID	Severity	Temuan	File	Dampak
FW-A07	REND AH	Timer Jitter pada NonBlockingTimer	NonBlockingTimer.h: 28-32	Setiap cycle mengakumulasi hingga 1ms jitter. Dampak signifikan pada timer high-frequency (PIR 200ms).

Tabel 4: Temuan Arsitektur dan Kualitas Kode Firmware

3. Audit Backend Google Apps Script

3.1 Gambaran Umum Backend

Backend SEMS diimplementasikan menggunakan Google Apps Script (GAS) sebagai REST API middleware antara firmware ESP32 dan frontend Next.js, dengan Google Spreadsheet sebagai basis data. Arsitektur ini terdiri dari 13 file .gs yang mencakup: routing (Code.gs), konfigurasi (Config.gs), autentikasi (Auth.gs), API handler untuk device, telemetry, config, alarm, schedule, rule, dan user, serta email notification, data cleanup, dan sheet setup. Backend mengimplementasikan RBAC (Role-Based Access Control) dengan tiga level peran: admin, technician, dan viewer.

Secara keseluruhan, backend menunjukkan organisasi kode yang cukup baik dengan pemisahan concerns yang jelas antara routing, handler API, autentikasi, dan utilitas. Terdapat beberapa fitur positif termasuk rate limiting pada login (5 percobaan per 15 menit), rate limiting email (1 per tipe per 30 menit + batas 20/hari), hashing password dengan salt, dan deduplikasi telemetry. Namun, terdapat beberapa kerentanan keamanan kritis yang perlu segera ditangani.

3.2 Temuan Keamanan Backend

ID	Severity	Temuan	File	Dampak
BE-S01	KRITIS	Semua Endpoint GET Tanpa Autentikasi	Code.gs:105-190	Semua GET route langsung ke handler tanpa cek auth. Data sensitif (telemetry, devices, users, config termasuk device_token) terbuka.

ID	Severity	Temuan	File	Dampak
BE-S 02	KRITIS	Token Terekspose via URL Query Parameter	Code.gs:310-331	Fallback token dari ? token=... muncul di browser history, logs, referrer headers. Anti-pattern yang berbahaya.
BE-S 03	KRITIS	Google Sheets Formula Injection	DeviceAPI, ConfigAPI, AlarmAPI, TelemetryAPI	String user-controlled ditulis langsung ke sel tanpa sanitasi. Mulai dengan "=" dieksekusi sebagai formula.
BE-S 04	KRITIS	Default Credentials di Source Code	Config.gs:201, SetupSheets.gs:319-343	Password default "changeme_immediately" untuk admin dan technician. Tidak ada mekanisme force change password.
BE-S 05	KRITIS	Timing-Safe Comparison Ada tapi Tidak Pernah Dipakai	Auth.gs:395-402 vs 58, 362	Fungsi compareHashes_() terdefinisi tapi perbandingan token/hash tetap menggunakan operator !=.
BE-S 06	TINGGI	Tidak Ada Rate Limiting pada POST Endpoints	Code.gs routePost_	Hanya login yang memiliki rate limiting. Telemetry spam dapat mengisi sheet (maks 5M cells), relay rapid toggle dapat merusak hardware.
BE-S 07	TINGGI	Single Shared Device Token - Tanpa Identitas Perangkat	Auth.gs:22-67	Semua ESP32 berbagi satu token. Tidak ada audit trail perangkat, tidak dapat revoke akses satu perangkat.
BE-S 08	TINGGI	Cleanup Berjalan di Setiap Telemetry POST	TelemetryAPI.gs:129-133	Sheet penuh (170K+ baris) di-scan setiap 15 detik. Execution time meningkat seiring akumulasi data. Time bomb.

ID	Severity	Temuan	File	Dampak
BE-S09	SEDANG	ConfigAPI Mengekspos Device Token di GET Response	ConfigAPI.gs:66-71	GET /api/config/get mengembalikan semua config termasuk device_token. Sensitif terutama karena GET tanpa auth.
BE-S10	SEDANG	Password Hashing Menggunakan SHA-256 Tanpa Key Stretching	Auth.gs:334-343	SHA-256 fast hash, salt hanya 16 char dari UUID. Rentan brute-force. GAS tidak mendukung bcrypt.

Tabel 5: Temuan Keamanan Backend

3.3 Temuan Performa dan Arsitektur

Dari sisi performa, masalah paling signifikan adalah mekanisme data cleanup yang berjalan pada setiap POST telemetry. Fungsi runDataCleanup() melakukan loading dan scanning seluruh sheet telemetry_history dan alarms pada setiap pengiriman data (interval 15 detik). Dengan 30 hari riwayat pada interval 15 detik, ini berpotensi 170.000+ baris yang harus dibaca dan di-scan per request. GAS memiliki batas eksekusi 6 menit dan memory 512MB, sehingga kinerja akan menurun secara progresif seiring bertambahnya data, dan akhirnya menyebabkan kegagalan telemetry.

Selain itu, penggunaan LockService.getScriptLock() secara global pada semua operasi write menyebabkan bottleneck. Dengan interval telemetry 15 detik dan operasi frontend yang bersamaan, timeout lock akan menjadi sering terjadi (lock wait 5 detik, mengembalikan HTTP 503). Lock contention ini akan menyebabkan pengalaman "Server busy" yang sering bagi pengguna frontend ketika ESP32 aktif mengirim data.

ID	Severity	Temuan	File	Dampak
BE-P01	TINGGI	Lock Contention - Single Global Lock Semua Operasi	Semua API files	Lock wait 5 detik. Frontend sering mendapat 503 ketika ESP32 aktif mengirim telemetry.

ID	Severity	Temuan	File	Dampak
BE-P02	SEDANG	CacheService TTL Maks 6 Jam - Session Abadi	Auth.gs:176-177	Code set TTL 24 jam tapi GAS batas 6 jam. Sliding window refresh membuat session tidak pernah expired.
BE-P03	RENDAH	Semua Session Di-invalidate Saat Satu User Diubah	UserAPI.gs:359-372	Perubahan role/active satu user memaksa semua user logout. Terlalu agresif.
BE-P04	RENDAH	Tidak Ada API Versioning	Code.gs semua route	Semua route tanpa versi (/api/telemetry bukan /api/v1/telemetry). Breaking changes akan mengganggu semua client.
BE-P05	RENDAH	Path Override via POST Body	Code.gs:50-54	Router membaca path dari request body jika pathInfo kosong. Routing manipulation potensial.

Tabel 6: Temuan Performa dan Arsitektur Backend

4. Audit Frontend Next.js / Vercel

4.1 Gambaran Umum Frontend

Frontend SEMS dibangun menggunakan Next.js 16.1.1 dengan React 19, TypeScript 5, dan Tailwind CSS 4. Aplikasi ini di-deploy ke Vercel dan mengimplementasikan Progressive Web App (PWA) dengan service worker, manifest, dan ikon. Arsitektur menggunakan App Router dengan direktori pages yang terstruktur, API routes sebagai proxy ke backend GAS, dan komponen UI berbasis shadcn/ui (25+ Radix UI primitives). Fitur dashboard meliputi: live overview, manajemen perangkat, analitik energi, alarm, automation/rules, schedules, load shedding, battery health, history, logs, notifications, user management, dan settings.

4.2 Temuan Keamanan Frontend

ID	Severity	Temuan	File	Dampak
FE-S01	KRITIS	Auth Token Lewat URL Query Parameter	api/sems/route.ts:56-59	Token GAS disisipkan sebagai ?token=... di URL. Muncul di server logs, browser history, referrer headers.
FE-S02	KRITIS	Token di localStorage Tanpa Expiry/Integrity	login/page.tsx:51-53	Auth token dan user object (termasuk role: admin) disimpan di localStorage tanpa expiry, signature, atau integrity check.
FE-S03	KRITIS	Cookie Middleware "sems-auth=1" Trivially Forgible	login/page.tsx:57, middleware.ts:11-15	Cookie statik "sems-auth=1" dapat dipalsukan via DevTools. Middleware protection sama sekali tidak efektif.
FE-S04	TINGGI	Tidak Ada Proteksi CSRF pada API Routes	api/sems/route.ts	POST endpoints untuk device control, rule creation, user creation tanpa CSRF token validation.
FE-S05	TINGGI	Role User Ditentukan dari Client-Side Data	useSemsAuth.ts:24-33	Role admin/technician/viewer ditentukan dari localStorage, bukan validasi server. UI admin terbuka tanpa validasi.
FE-S06	TINGGI	Tidak Ada Rate Limiting pada Login	login/page.tsx, api/sems/route.ts	Brute force credential attack melalui proxy frontend ke backend GAS.
FE-S07	SEDANG	Cookie Tanpa Secure/HttpOnly Flags	login/page.tsx:57	Cookie "sems-auth=1" tanpa flag Secure dan HttpOnly. Dapat dikirim melalui HTTP dan diakses JavaScript.
FE-S08	SEDANG	NEXTAUTH_SECRET Dependency Tanpa Validasi	lib/auth.ts:107	Jika NEXTAUTH_SECRET tidak diset, NextAuth dapat fallback ke generated secret yang berubah tiap deployment.

Tabel 7: Temuan Keamanan Frontend

4.3 Temuan Kualitas Kode dan Performa

Frontend memiliki beberapa masalah kualitas kode yang signifikan. Pertama, bundle client-side sangat besar dengan lebih dari 80 dependensi termasuk recharts (~200KB), framer-motion (~100KB), @dnd-kit (~50KB), dan react-syntax-highlighter (~100KB+). Tidak ada dynamic imports atau lazy loading yang digunakan di mana pun. @tanstack/react-query terinstal tapi tidak pernah digunakan - semua halaman dashboard mengimplementasikan manual useState + useEffect + useCallback pattern, sehingga tidak ada cache invalidation, deduplication, retry logic, atau optimistic updates.

Kedua, terdapat sistem autentikasi ganda yang saling bertumpang tindih: custom SEMS auth (direct GAS login ke localStorage) dan NextAuth (JWT-based session dengan SessionProvider). TokenSync.tsx menjembatani keduanya, tetapi menambah kompleksitas tanpa manfaat yang jelas. Ketiga, TypeScript build errors di-suppress dengan ignoreBuildErrors: true, sehingga potensi type-safety bugs dapat mencapai production. Keempat, file halaman seperti analytics (942 baris) dan automation (1008 baris) terlalu besar dengan helper functions, sub-components, state management, dan rendering dalam satu file.

ID	Severity	Temuan	File	Dampak
FE-P01	TINGGI	Bundle Besar - Tidak Ada Code Splitting	package.json, semua pages	80+ dependensi tanpa lazy loading. Estimasi bundle >500KB gzipped.
FE-P02	TINGGI	React Query Terinstal Tapi Tidak Digunakan	package.json:50, semua dashboard pages	Manual data fetching tanpa caching, deduplication, retry, atau optimistic updates.
FE-P03	SEDANG	Dual Auth System - Custom SEMS + NextAuth	useSemsAuth.ts, TokenSync.tsx	Dua sistem autentikasi paralel menambah kompleksitas tanpa manfaat jelas. NextAuth tampak legacy.
FE-P04	SEDANG	PWA Cache Strategy Minimal	public/sw.js	Hanya 4 static assets di-cache. Application bundle (JS/CSS) tidak di-pre-cache untuk offline.
FE-P05	SEDANG	API Fallback ke Homepage saat Offline	public/sw.js:47-53	API call gagal offline mengembalikan HTML homepage, menyebabkan JSON parse error.

ID	Severity	Temuan	File	Dampak
FE-P-06	REND AH	Prisma Schema Default Boilerplate Tidak Digunakan	prisma/schema.prisma	User dan Post models dari Prisma init. Data asli di Google Sheets. Dead code.
FE-P-07	REND AH	React Strict Mode Disabled	next.config.ts:9	Strict mode dimatikan. Potensi side effect issues di useEffect tidak terdeteksi saat development.

Tabel 8: Temuan Kualitas Kode dan Performa Frontend

5. Analisis Integrasi Antar Komponen

5.1 Alur Data End-to-End

Alur data SEMS dimulai dari firmware ESP32 yang membaca sensor secara periodik (interval polling sensor), memproses data melalui sensor engine, menyimpannya dalam struct SensorData global, lalu mengirimkan telemetry ke backend GAS melalui HTTPS POST request. Backend GAS menerima telemetry, menulis ke Google Spreadsheet, memeriksa kondisi alarm, dan mengirimkan notifikasi email jika diperlukan. Frontend Next.js mengambil data dari backend GAS melalui API proxy route (/api/sems) yang meneruskan request ke GAS web app URL.

Proses kontrol beban berjalan dalam dua arah: frontend mengirim perintah kontrol relay melalui GAS, yang kemudian disimpan di spreadsheet. Firmware melakukan config sync secara periodik untuk mengambil konfigurasi terbaru (termasuk state relay) dari backend. Selain itu, firmware juga mengimplementasikan automation engine lokal yang dapat mengontrol relay berdasarkan kondisi sensor secara otonom, memberikan kemampuan kontrol lokal meskipun koneksi internet terputus.

5.2 Temuan Integrasi

ID	Severity	Temuan	File	Dampak
INT-01	KRITIS	Token di URL pada Seluruh Alur Komunikasi	Firmware + Frontend + Backend	Baik firmware dan frontend mengirim token via URL parameter. Token muncul di GAS logs, Vercel logs, dan browser history.

ID	Severity	Temuan	File	Dampak
INT-02	TINGGI	Tidak Ada Request Correlation ID	Semua komponen	Tidak ada cara mengkorelasikan telemetry POST dengan cleanup atau alarm notification. Debugging lintas komponen sangat sulit.
INT-03	TINGGI	Single Point of Failure pada API Proxy	api/sems/route.ts	Semua API call melalui satu proxy route. Jika proxy gagal (15s timeout), seluruh aplikasi offline tanpa fallback.
INT-04	SEDANG	API Proxy Menerima Arbitrary Path	api/sems/route.ts:28-29	Proxy meneruskan path apapun ke GAS tanpa validation atau allowlist. Open relay ke GAS URL.
INT-05	SEDANG	Dual Database - Spreadsheet dan Prisma Tidak Terpakai	prisma/schema.prisma	Prisma schema ada tapi tidak terpakai. Seluruh data di Google Sheets. Dead code membingungkan developer baru.
INT-06	SEDANG	Bahasa UI Tidak Konsisten	Semua halaman frontend	Campuran Indonesia dan Inggris dalam UI, komentar kode, error messages. Pengalaman pengguna terpecah.

Tabel 9: Temuan Integrasi Antar Komponen

6. Rekomendasi Perbaikan

6.1 Aksi Segera (0-7 Hari)

Berikut adalah rekomendasi perbaikan yang harus dilakukan secara **immediate** sebelum sistem digunakan di lingkungan produksi. Penundaan pada item-item ini dapat mengakibatkan kompromi keamanan, kerusakan perangkat keras, atau kegagalan sistem.

Rekomendasi	Komponen	Detail
Ganti SEMUA kredensial hardcoded	Firmware	Ubah WiFi password, API token, OTA password. Implementasikan mekanisme provisioning yang aman (BLE setup atau WiFi Provisioning).
Implementasikan tindakan protektif di Safety Engine	Firmware	Overvoltage harus memutus relay charging. Undervoltage harus memaksa emergency load shed. Overcurrent harus trip contactor. Ini adalah masalah keselamatan perangkat keras.
Tambahkan autentikasi pada semua GET routes	Backend	Semua GET endpoint di routeGet_() harus memeriksa auth sebelum mengembalikan data. Terutama GET /api/config/get yang mengekspos device token.
Ganti cookie "sems-auth=1" dengan signed token	Frontend	Middleware harus memvalidasi token HMAC atau JWT secara server-side. Cookie statik yang dapat dipalsukan harus dihapus.
Sanitasi input sebelum menulis ke Google Sheets	Backend	Prefix semua string yang dikontrol user dengan tanda petik tunggal (') atau gunakan type coercion untuk mencegah formula injection.
Hapus kode modular yang tidak dapat dikompilasi	Firmware	Selesaikan refactoring atau hapus file sensors/, engines/, utils/ yang tidak kompatibel. Dual codebase menjamin divergensi.

Tabel 10: Rekomendasi Aksi Segera

6.2 Prioritas Tinggi (1-4 Minggu)

Item-item berikut harus ditangani dalam jangka pendek setelah aksi segera selesai. Mereka mengurangi risiko keamanan dan meningkatkan keandalan sistem secara signifikan.

Rekomendasi	Komponen	Detail
Implementasikan per-device token	Backend + Firmware	Ganti shared token dengan identitas perangkat unik. Aktifkan audit trail dan kemampuan revoke akses per perangkat.
Tambahkan input validation pada telemetry	Backend	Validasi range numerik (voltage, current, temperature) sebelum menulis ke spreadsheet. Cegah data anomali.
Implementasikan overcurrent dan temperature monitoring	Firmware	Aktifkan pengecekan SAFETY_OVERCURRENT_A dan SAFETY_MAX_TEMP_C di firmware produksi (.ino).

Rekomendasi	Komponen	Detail
Hapus token dari URL query parameter	Firmware + Frontend	Gunakan request body untuk POST atau implementasikan google.script.run client API. Hapus fallback ?token=...
Gunakan React Query untuk data fetching	Frontend	Migrasi dari manual useState/useEffect ke @tanstack/react-query (sudah terinstal). Aktifkan caching dan deduplication.
Implementasikan tiered load shedding	Firmware	Ganti shedding biner dengan priority-based shedding sesuai TIER1/TIER2/TIER3. Non-esensial duluan, kenyamanan terakhir.
Kurangi frekuensi EEPROM write atau gunakan NVS	Firmware	EEPROM wear ~70 hari pada rate saat ini. Gunakan NVS atau kurangi interval simpan SOC.

Tabel 11: Rekomendasi Prioritas Tinggi

6.3 Prioritas Menengah (1-3 Bulan)

Item-item berikut meningkatkan kualitas, maintainability, dan pengalaman pengguna tetapi tidak bersifat mendesak. Mereka dapat direncanakan dalam roadmap pengembangan jangka menengah.

Rekomendasi	Komponen	Detail
Konsolidasi sistem autentikasi	Frontend	Hapus NextAuth jika tidak digunakan. Gunakan hanya custom SEMS auth dengan perbaikan security.
Hapus dependensi dan dead code	Frontend	Hapus Prisma schema, zustand, @mdxeditor/editor, dan dependensi unused lainnya. Kurangi bundle size.
Implementasikan PWA caching yang proper	Frontend	Pre-cache application bundle untuk offline support. Tambahkan background sync untuk offline actions.
Standardisasi bahasa UI	Frontend	Pilih satu bahasa (Indonesia atau Inggris) untuk seluruh UI, bukan campuran keduanya.
Implementasikan PBKDF2 untuk password hashing	Backend	Ganti SHA-256 single-round dengan PBKDF2 multi-round (GAS tidak mendukung bcrypt).
Tambahkan request correlation ID	Semua komponen	Propagasikan unique ID dari firmware ke GAS ke frontend untuk debugging lintas komponen.

Rekomendasi	Komponen	Detail
Throttle cleanup di backend	Backend	Jalankan cleanup setiap N-th telemetry POST, bukan setiap kali. Kurangi lock contention dan execution time.
Tambahkan API versioning	Backend	Migrasi ke /api/v1/... untuk memungkinkan breaking changes tanpa mengganggu client lama.
Aktifkan TypeScript build errors	Frontend	Set ignoreBuildErrors: false. Perbaiki type errors untuk mencegah bugs reach production.

Tabel 12: Rekomendasi Prioritas Menengah

7. Kesimpulan

Sistem SEMS menunjukkan arsitektur yang inovatif dengan integrasi ESP32, Google Apps Script, Spreadsheet, dan Next.js/Vercel sebagai solusi IoT monitoring energi dengan biaya infrastruktur yang minimal. Fitur-fitur seperti automation engine lokal, load shedding bertingkat (meskipun belum lengkap), RBAC multi-level, dan email notification dengan rate limiting menunjukkan pemikiran desain yang matang dalam beberapa aspek.

Namun, audit ini mengungkapkan bahwa sistem ini **belum siap untuk deployment produksi** dalam kondisi saat ini. Terdapat 16 temuan KRITIS yang mencakup: kredensial hardcoded di firmware, Safety Engine yang tidak mengambil tindakan protektif terhadap kondisi baterai berbahaya, seluruh endpoint GET backend tanpa autentikasi, middleware frontend yang trivially bypassable, dan potensi formula injection di Google Sheets. Temuan-temuan ini, jika dieksploitasi, dapat menyebabkan kompromi data, pengambilalihan perangkat, kerusakan perangkat keras baterai, atau bahkan risiko kebakaran.

Rekomendasi utama adalah memprioritaskan perbaikan pada enam item aksi segera (Bagian 6.1) sebelum sistem digunakan di lingkungan nyata. Dengan perbaikan yang tepat pada aspek keamanan, keselamatan perangkat keras, dan kualitas kode, SEMS memiliki potensi untuk menjadi platform manajemen energi yang andal dan aman. Tim pengembangan disarankan untuk mengadopsi pendekatan security-by-design dan safety-by-design dalam iterasi pengembangan selanjutnya, termasuk implementasi code review, pengujian unit untuk logika safety-critical, dan security testing berkala.